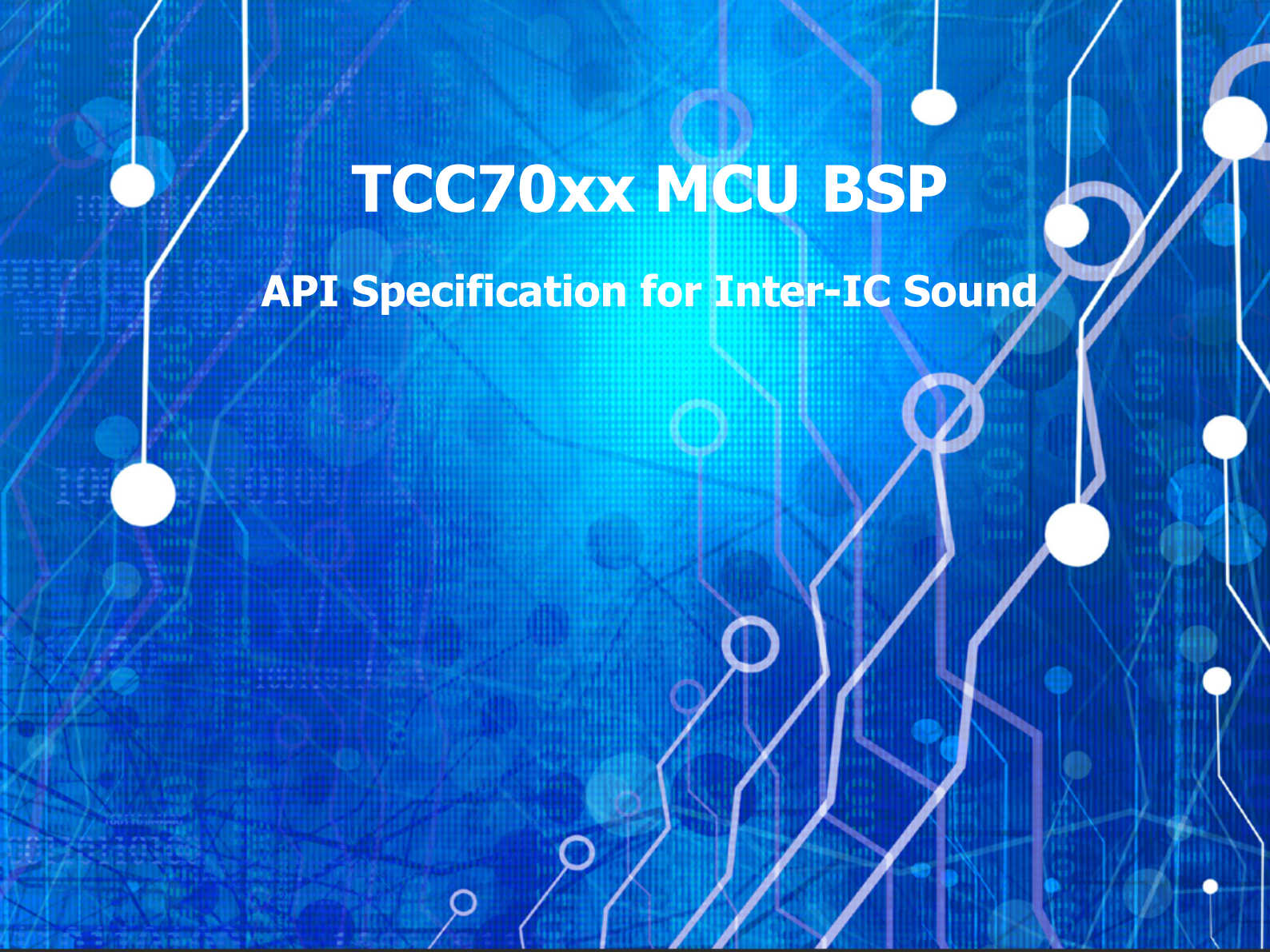




TCC70xx MCU BSP

API Specification for Inter-IC Sound

Rev. 0.10 [A]

2021-09-03

Preliminary version

※ The information in this document is subject to change without notice and should not be construed as a commitment by Telechips Inc.

Kindly visit www.telechips.com for more information.

© 2021 Telechips Inc. All rights reserved.

TABLE OF CONTENTS

Contents

1	Introduction	3
2	Terms and Abbreviations	3
3	Source Files.....	4
3.1	Location of Source Files.....	4
3.2	Simple Description of Source Files.....	4
4	Enumerations	5
4.1	I2SStatus_t.....	5
4.2	I2SHwCh_t	6
4.3	I2SMode_t.....	7
4.4	I2SFormat_t.....	8
4.5	I2SChannel_t.....	9
4.6	I2SLRmode_t.....	10
4.7	I2SSampleRate_t.....	11
4.8	I2SBitDepth_t	12
4.9	I2SBclkDiv_t.....	13
4.10	I2SMclkDiv_t.....	14
4.11	I2SRetCode_t.....	15
5	Structures	16
5.1	I2SConfig_t.....	16
6	APIs	17
6.1	I2S_SetGpiofunction	17
6.2	I2S_SetClock.....	18
6.3	I2S_TxADmaSetting	19
6.4	I2S_DaifSetting	20
6.5	I2S_SetTransferSize	21
6.6	I2S_GetTransferSize.....	22
6.7	I2S_FifoClear	23
6.8	I2S_DmaReset	24
6.9	I2S_SWReset	25
6.10	I2S_DAIEnable	26
6.11	I2S_DAIDisable	27
6.12	I2S_DMAEnable.....	28
6.13	I2S_DMADisable	29
6.14	I2S_Enable	30
6.15	I2S_Disable.....	31
6.16	I2S_GetTxDMAStatus.....	32
6.17	I2S_RepeatMode	33
6.18	I2S_SetTxDaSar	34
6.19	I2S_GetTxDaCsar	35
7	How to use I2S Driver	36
7.1	Initialization	36
7.2	Ready.....	36
7.3	Play.....	37
8	References	38
9	Revision History	39
	Rev. 0.10: 2021-09-03	39

Figures

Figure 3.1 Location of Source Files	4
---	---

Tables

Table 2.1 Terms and Abbreviation	3
--	---

1 INTRODUCTION

This document describes the Inter-IC Sound device driver (hereinafter referred to as I2S driver) of TCC70xx MCU BSP.

2 TERMS AND ABBREVIATIONS

Table 2.1 Terms and Abbreviation

I2S	Inter-IC Sound
DAIF	Digital Audio Interface
ADMA	Audio Direct Memory Access
MCLK	Master clock
BCLK	Bit clock

3 SOURCE FILES

3.1 Location of Source Files

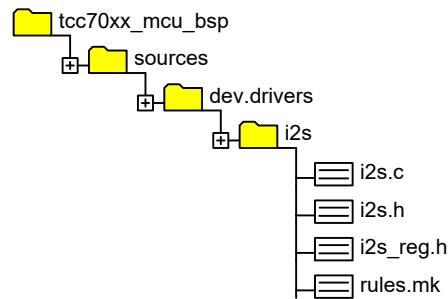


Figure 3.1 Location of Source Files

3.2 Simple Description of Source Files

- i2s.c
 - I2S driver interface source
- i2s.h
 - I2S driver interface header
- i2s_reg.h
 - Define header of DAIF, ADMA registers
- rules.mk
 - Rules used for build

4 ENUMERATIONS

4.1 I2SStatus_t

Enumeration variables for I2S status

Declaration

```
typedef enum I2SStatus
{
    I2S_ERROR      = 0,
    I2S_DONE,
    I2S_READY,
    I2S_PLAYBACK,
    I2S_PAUSE,
    I2S_RESUME,
    I2S_STOP
} I2SStatus_t;
```

Description

Value	Description
<i>I2S_ERROR</i>	<i>Fail to operate I2S driver</i>
<i>I2S_DONE</i>	<i>Standby</i>
<i>I2S_READY</i>	<i>Initialization of I2S setting is done.</i>
<i>I2S_PLAYBACK</i>	<i>Play audio streaming</i>
<i>I2S_PAUSE</i>	<i>Pause audio streaming</i>
<i>I2S_RESUME</i>	<i>Resume audio streaming</i>
<i>I2S_STOP</i>	<i>Playback is over. Disable DAIF and ADMA</i>

Remark

None

4.2 I2SHwCh_t

Enumeration variables for I2S hardware channel

Declaration

```
typedef enum I2SHwCh
{
    I2S_CH0    = 0,
    I2S_CH1,
    I2S_CH2
} I2SHwCh_t;
```

Description

<i>Value</i>	<i>Description</i>
<i>I2S_CH0</i>	<i>Channel 0</i>
<i>I2S_CH1</i>	<i>Channel 1</i>
<i>I2S_CH2</i>	<i>Channel 2</i>

Remark

None

4.3 I2SMode_t

Enumeration variables for I2S mode

Declaration

```
typedef enum I2SMode
{
    I2S_MASTER_MODE    = 0,
    I2S_SLAVE_MODE     = 1
} I2SMode_t;
```

Description

Value	Description
<i>I2S_MASTER_MODE</i>	<i>Master mode</i> <i>Generate MCLK, BCLK, LRCK inside of TCC70xx</i>
<i>I2S_SLAVE_MODE</i>	<i>Slave mode</i> <i>Get MCLK, BCLK, LRCK from outside of TCC70xx</i>

Remark

None

4.4 I2SFormat_t

Enumeration variables for I2S format

Declaration

```
typedef enum I2SFormat
{
    I2S_FORMAT_I2S          = 1,
    I2S_FORMAT_RIGHTJ,
    I2S_FORMAT_LEFTJ
} I2SFormat_t;
```

Description

<i>Value</i>	<i>Description</i>
<i>I2S_FORMAT_I2S</i>	<i>I2S mode</i>
<i>I2S_FORMAT_RIGHTJ</i>	<i>Right Justified mode = Most Significant Bit</i>
<i>I2S_FORMAT_LEFTJ</i>	<i>Left Justified mode = Least Significant Bit</i>

Remark

None

4.5 I2SChannel_t

Enumeration variables for the number of I2S channels

Declaration

```
typedef enum I2SChannel
{
    I2S_MONO      = 0,
    I2S_STEREO    = 1
} I2SChannel_t;
```

Description

Value	Description
I2S_MONO	Single audio channel
I2S_STEREO	Two audio channels

Remark

None

4.6 I2SLRmode_t

Enumeration variables for I2S LR Mode

Declaration

```
typedef enum I2SLRmode
{
    I2S_LRMODE_OFF    = 0,
    I2S_LRMODE_ON     = 1
} I2SLRmode_t;
```

Description

Value	Description
<i>I2S_LRMODE_OFF</i>	<i>Left channel data and right channel data have the same source/destination address.</i>
<i>I2S_LRMODE_ON</i>	<i>Left channel data and right channel data have independent source/destination address.</i>

Remark

None

4.7 I2SSampleRate_t

Enumeration variables for I2S Sampling Rate

Declaration

```
typedef enum I2SSampleRate
{
    I2S_SAMPLE_RATE_8000      = 0,
    I2S_SAMPLE_RATE_16000,
    I2S_SAMPLE_RATE_24000,
    I2S_SAMPLE_RATE_32000,
    I2S_SAMPLE_RATE_44100,
    I2S_SAMPLE_RATE_48000,
    I2S_SAMPLE_RATE_64000,
    I2S_SAMPLE_RATE_96000,
    I2S_SAMPLE_RATE_128000,
    I2S_SAMPLE_RATE_192000
} I2SSampleRate_t;
```

Description

Value	Description
<i>I2S_SAMPLE_RATE_8000</i>	<i>Audio sampling rate 8000</i>
<i>I2S_SAMPLE_RATE_16000</i>	<i>Audio sampling rate 16000</i>
<i>I2S_SAMPLE_RATE_24000</i>	<i>Audio sampling rate 24000</i>
<i>I2S_SAMPLE_RATE_32000</i>	<i>Audio sampling rate 32000</i>
<i>I2S_SAMPLE_RATE_44100</i>	<i>Audio sampling rate 44100</i>
<i>I2S_SAMPLE_RATE_48000</i>	<i>Audio sampling rate 48000</i>
<i>I2S_SAMPLE_RATE_64000</i>	<i>Audio sampling rate 64000</i>
<i>I2S_SAMPLE_RATE_96000</i>	<i>Audio sampling rate 96000</i>
<i>I2S_SAMPLE_RATE_128000</i>	<i>Audio sampling rate 128000</i>
<i>I2S_SAMPLE_RATE_192000</i>	<i>Audio sampling rate 192000</i>

Remark

None

4.8 I2SBitDepth_t

Enumeration variables for I2S Bit Depth

Declaration

```
typedef enum I2SBitDepth
{
    I2S_BIT_DEPTH_16  = 16,
    I2S_BIT_DEPTH_24  = 24,
} I2SBitDepth_t;
```

Description

<i>Value</i>	<i>Description</i>
<i>I2S_BIT_DEPTH_16</i>	<i>16 bits for each sample</i>
<i>I2S_BIT_DEPTH_24</i>	<i>24 bits for each sample</i>

Remark

None

4.9 I2SBclkDiv_t

Enumeration variables for generating I2S DAIF Frame clock by dividing DAIF bit clock

Declaration

```
typedef enum I2SBclkDiv
{
    I2S_BCLK_DIV_32      = 0,
    I2S_BCLK_DIV_48      = 1,
    I2S_BCLK_DIV_64      = 2,
    I2S_BCLK_RESERVED    = 3
} I2SBclkDiv_t;
```

Description

Value	Description
I2S_BCLK_DIV_32	Bit clock divided by 32
I2S_BCLK_DIV_48	Bit clock divided by 48
I2S_BCLK_DIV_64	Bit clock divided by 64
I2S_BCLK_RESERVED	Reserved

Remark

None

4.10 I2SMclkDiv_t

Enumeration variables for generating I2S DAIF Bit clock by dividing DAIF master clock.

Declaration

```
typedef enum I2SMclkDiv
{
    I2S_MCLK_DIV_4    = 0,
    I2S_MCLK_DIV_6    = 1,
    I2S_MCLK_DIV_8    = 2,
    I2S_MCLK_DIV_16   = 3,
    I2S_MCLK_DIV_24   = 4,
    I2S_MCLK_DIV_32   = 5,
    I2S_MCLK_DIV_48   = 6,
    I2S_MCLK_DIV_64   = 7
} I2SMclkDiv_t;
```

Description

Value	Description
I2S_MCLK_DIV_4	Master clock divided by 4
I2S_MCLK_DIV_6	Master clock divided by 6
I2S_MCLK_DIV_8	Master clock divided by 8
I2S_MCLK_DIV_16	Master clock divided by 16
I2S_MCLK_DIV_24	Master clock divided by 24
I2S_MCLK_DIV_32	Master clock divided by 32
I2S_MCLK_DIV_48	Master clock divided by 48
I2S_MCLK_DIV_64	Master clock divided by 64

Remark

None

4.11 I2SRetCode_t

Enumeration variables for I2S operation result

Declaration

```
typedef enum I2SRetCode
{
    I2S_RET_SUCCESS    = 0,
    I2S_RET_FAILED     = 1
} I2SRetCode_t;
```

Description

<i>Value</i>	<i>Description</i>
<i>I2S_RET_SUCCESS</i>	<i>Success</i>
<i>I2S_RET_FAILED</i>	<i>Failure</i>

Remark

None

5 STRUCTURES

5.1 I2SConfig_t

This structure includes I2S Configuration.

Declaration

```
typedef struct I2SConfig
{
    I2SStatus_t      i2sStatus;
    I2SHwCh_t        i2sHwCh;
    I2SMode_t         i2sMode;
    I2SFormat_t       i2sFormat;
    I2SChannel_t      i2sNumCh;
    I2SLRmode_t       i2sLRmode;
    I2SSampleRate_t   i2sSampleRate;
    I2SBitDepth_t     i2sBitPerSample;
    I2SBclkDiv_t       i2sBclkDiv;
    I2SMclkDiv_t       i2sMclkDiv;
    uint32            i2sDataSize;
    uint32 *           i2sTxbuffer;
} I2SConfig_t;
```

Description

Value	Description
i2sStatus	Confirm I2S status. Refer to I2SStatus_t .
i2sHwCh	I2S hardware channel. Refer to I2SHwCh_t .
i2sMode	I2S Master or Slave mode
i2sFormat	Digital audio data format. Refer to I2SFormat_t .
i2sNumCh	The number of Audio output Channels. Refer to I2SChannel_t .
i2sLRmode	LRmode Enable/Disable. Refer to I2SLRmode_t .
i2sSampleRate	Digital Audio sampling rate. Refer to I2SSampleRate_t .
i2sBitPerSample	The number of bits of information in each sample. Refer to I2SBitDepth_t .
i2sBclkDiv	Make frame clock dividing bit clock. Refer to I2SBclkDiv_t .
i2sMclkDiv	Make bit clock dividing master clock. Refer to I2SMclkDiv_t .
i2sDataSize	Audio data size for successfully terminating audio play
i2sTxbuffer	Audio data tx buffer address

Remark

None

6 APIs

6.1 I2S_SetGpiofunction

Sets GPIO function as I2S Interface.

Declaration

```
I2SRetCode_t I2S_SetGpiofunction  
(  
    I2SConfig_t * psI2sConf  
);
```

Parameters

<i>Value</i>	<i>Description</i>
<i>psI2sConf</i>	<i>[in] I2S configuration</i>

Return

<i>Value</i>	<i>Description</i>
<i>I2S_RET_SUCCESS</i>	<i>Success</i>
<i>I2S_RET_FAILED</i>	<i>Failure</i>

Remark

None

6.2 I2S_SetClock

Sets master clock (MCLK), bit clock (BCLK), frame clock (LRCK), and filter clock frequency.

Declaration

```
I2SRetCode_t I2S_SetClock  
(  
    I2SConfig_t * psI2sConf  
);
```

Parameters

<i>Value</i>	<i>Description</i>
<i>psI2sConf</i>	<i>[in]</i> I2S configuration

Return

<i>Value</i>	<i>Description</i>
<i>I2S_RET_SUCCESS</i>	<i>Success</i>
<i>I2S_RET_FAILED</i>	<i>Failure</i>

Remark

None

6.3 I2S_TxAdmaSetting

Sets AUDIO DMA related register.

Declaration

```
void I2S_AdmaSetting  
(  
    I2SConfig_t * psI2sConf  
);
```

Parameters

Value	Description
<i>psI2sConf</i>	<i>[in] I2S configuration</i>

Return

Value	Description
<i>I2S_RET_SUCCESS</i>	<i>Success</i>
<i>I2S_RET_FAILED</i>	<i>Failure</i>

Remark

None

6.4 I2S_DaifSetting

Sets Digital Audio Interface (DAIF) related register.

Declaration

```
void I2S_DaifSetting  
(  
    I2SConfig_t * psI2sConf  
);
```

Parameters

Value	Description
<i>psI2sConf</i>	<i>[in]</i> I2S configuration

Return

Value	Description
<i>I2S_RET_SUCCESS</i>	<i>Success</i>
<i>I2S_RET_FAILED</i>	<i>Failure</i>

Remark

None

6.5 I2S_SetTransferSize

Sets the SINC, SMASK, ST_TCOUNT bit for setting transfer size.

Declaration

```
void I2S_SetTransferSize
(
    I2SConfig_t * psI2sConf
);
```

Parameters

<i>Value</i>	<i>Description</i>
<i>psI2sConf</i>	<i>[in] I2S configuration</i>

Return

<i>Value</i>	<i>Description</i>
<i>I2S_RET_SUCCESS</i>	<i>Success</i>
<i>I2S_RET_FAILED</i>	<i>Failure</i>

Remark

None

6.6 I2S_GetTransferSize

Returns the Transfer Size calculated by register value.

Declaration

```
uint32 I2S_GetTransferSize
(
    void
);
```

Parameters

None

Return

Value	Description
ui_UnitSize	Transfer data size

Remark

None

6.7 I2S_FifoClear

Clears transmitter FIFO buffer.

Declaration

```
void I2S_FifoClear
(
    uint32 uiClrBit
);
```

Parameters

Value	Description
uiClrBit	[in] I2S_DOUT

Return

None

Remark

None

6.8 I2S_DmaReset

Resets Audio DMA.

Declaration

```
void I2S_DmaReset
(
    uint32 uiRstBit
);
```

Parameters

Value	Description
uiRstBit	[in] I2S_DOUT

Return

None

Remark

None

6.9 I2S_SWReset

I2S Software Reset.

Declaration

```
void I2S_SWReset
(
    boolean    bReset
);
```

Parameters

Value	Description
bReset	SALEnabled, SALDisabled

Return

None

Remark

None

6.10 I2S_DAIEnable

Enables DAIF transmitter.

Declaration

```
void I2S_DAIEnable
(
    uint8 ucflag
);
```

Parameters

Value	Description
ucflag	[in] I2S_DOUT

Return

None

Remark

None

6.11 I2S_DAIDisable

Disables DAIF transmitter.

Declaration

```
void I2S_DAIDisable
(
    uint8 ucflag
);
```

Parameters

Value	Description
ucflag	[in] I2S_DOUT

Return

None

Remark

None

6.12 I2S_DMAEnable

Enables DMA Channel of DAIF transmitter.

Declaration

```
void I2S_DAIEnable
(
    uint8 ucflag
);
```

Parameters

Value	Description
ucflag	[in] I2S_DOUT

Return

None

Remark

None

6.13 I2S_DMADisable

Disables DMA channel of DAIF transmitter.

Declaration

```
void I2S_DAIDisable
(
    uint8 ucflag
);
```

Parameters

Value	Description
ucflag	[in] I2S_DOUT

Return

None

Remark

None

6.14 I2S_Enable

Calls I2S_DMAEnable and I2S_DAIEnable functions.

Declaration

```
void I2S_Enable
(
    uint8 ucflag
);
```

Parameters

Value	Description
ucflag	[in] I2S_DOUT

Return

None

Remark

None

6.15 I2S_Disable

Calls I2S_DMADisable and I2S_DAIDisable function.

Declaration

```
void I2S_Disable
(
    uint8 ucflag
);
```

Parameters

Value	Description
ucflag	[in] I2S_DOUT

Return

None

Remark

None

6.16 I2S_GetTxDMAStatus

Gets DMA Tx Channel Enable/Disable status.

Declaration

```
uint8 I2S_GetTxDMAStatus
(
    void
);
```

Parameters

None

Return

Value	Description
1	DAIF Tx DMA channel is enabled.
0	DAIF Tx DMA channel is terminated and disabled.

Remark

None

6.17 I2S_RepeatMode

Sets transmitter DMA Repeat mode.

Declaration

```
void I2S_RepeatMode
(
    uint8      ucflag,
    boolean    bEnable
);
```

Parameters

Value	Description
ucflag	[in] I2S_DOUT
bEnable	[in] SALEnabled/SALDisabled

Return

None

Remark

None

6.18 I2S_SetTxDaSar

Sets DAIF Tx Data Source Address.

Declaration

```
void I2S_SetTxDaSar
(
    uint32 uiTxDataSourceAddr
);
```

Parameters

Value	Description
uiTxDataSourceAddr	[in] DAIF Tx (Right) Data0 Source Address

Return

None

Remark

None

6.19 I2S_GetTxDaCsar

Gets DAIF Tx Data Source Address.

Declaration

```
uint32 I2S_GetTxDaCsar
(
    void
);
```

Parameters

None

Return

Value	Description
0 ~ 4,294,967,295	[out] DAIF Tx (Right) Data0 Source Address

Remark

None

7 HOW TO USE I2S DRIVER

7.1 Initialization

To use the I2S, you should set both ADMA and DAIF Registers. Initialize the I2S configuration as shown in the following code. Non-cacheable memory is used as audio buffer memory.

```
{
    ...
    I2SConfig_t    s_i2s_config;

    s_i2s_config.i2sStatus      = I2S_DONE;
    s_i2s_config.i2sHwCh        = I2S_CH0;
    s_i2s_config.i2sMode         = I2S_MASTER_MODE;
    s_i2s_config.i2sFormat       = I2S_FORMAT_I2S;
    s_i2s_config.i2sNumCh        = I2S_STEREO;
    s_i2s_config.i2sLRmode       = I2S_LRMODE_OFF;
    s_i2s_config.i2sSampleRate    = I2S_SAMPLE_RATE_32000;
    s_i2s_config.i2sBitPerSample = I2S_BIT_DEPTH_16;
    s_i2s_config.i2sBclkDiv       = I2S_BCLK_DIV_64;
    s_i2s_config.i2sMclkDiv       = I2S_MCLK_DIV_6;

    s_i2s_config.i2sDataSize      = sizeof(g_Audio_Sample);
    s_i2s_config.i2sTxbuffer       = AUDIO_TXBuffer;
    ...

    I2S_SWReset(SALEnabled);
    I2S_SWReset(SALDisabled);

    I2S_SetGpiofunction(&s_i2s_config);
    I2S_SetClock(&s_i2s_config);
    I2S_AdmaSetting(&s_i2s_config);
    I2S_DaifSetting(&s_i2s_config);
    I2S_FifoClear(I2S_DOUT);

    I2S_SetTransferSize(&s_i2s_config);

    s_i2s_config.i2sStatus = I2S_READY;
}
```

7.2 Ready

Copy audio data to audio buffer.

```
ui_TxUnitSize = (uint32)I2S_GetTransferSize();
AUDIO_Read((void *)ui_AuDataCurAddr, ui_TxUnitSize, (void *)ui_TxBufAddr);
```

7.3 Play

```
{
    ...
    s_i2s_config.i2sStatus = I2S_PLAYBACK;
    I2S_Enable(I2S_DOUT);
    ...
}

static void AUDIO_AppTask(void * pArg)
{
    ...

    if((I2S_GetTxDataCsar() >= ui_TxBufAddrHalf) && (uc_TxStatus == BUF2_DONE))
    {
        /* DMA Current Address is BUF2 area, Load BUF1 Data. */
        //mcu_printf("ui_AuDataCurAddr 0x%x \r\n", ui_AuDataCurAddr);
        AUDIO_Read((void *)ui_AuDataCurAddr, ui_TxUnitSize, (void *)ui_TxBufAddr);

        ui_AuDataCurAddr += ui_TxUnitSize;
        uc_TxStatus = BUF1_DONE;
    }

    if((I2S_GetTxDataCsar() >= ui_TxBufAddr) && (I2S_GetTxDataCsar() < ui_TxBufAddrHalf)
        && (uc_TxStatus == BUF1_DONE))
    {
        /* DMA Current Address is BUF1 area , Load BUF2 Data. */
        //mcu_printf("ui_AuDataCurAddr 0x%x \r\n", ui_AuDataCurAddr);
        AUDIO_Read((void *)ui_AuDataCurAddr, ui_TxUnitSize, (void *)ui_TxBufAddrHalf);

        ui_AuDataCurAddr += ui_TxUnitSize;
        uc_TxStatus = BUF2_DONE;
    }
}
```

8 REFERENCES

- [1] TCC70xx Full Specification
- [2] Contact Telechips for more details: sales@telechips.com

9 REVISION HISTORY

Rev. 0.10: 2021-09-03

- Preliminary version release

DISCLAIMER

All information and data contained in this material are without any commitment, are not to be considered as an offer for conclusion of a contract, nor shall they be construed as to create any liability. Any new issue of this material invalidates previous issues. Product availability and delivery are exclusively subject to our respective order confirmation form; the same applies to orders based on development samples delivered. By this publication, Telechips, Inc. does not assume responsibility for patent infringements or other rights of third parties that may result from its use.

Further, Telechips, Inc. reserves the right to revise this publication and to make changes to its content, at any time, without obligation to notify any person or entity of such revisions or changes.

No part of this publication may be reproduced, photocopied, stored on a retrieval system, or transmitted without the express written consent of Telechips, Inc.

This product is designed for general purpose, and accordingly customer be responsible for all or any of intellectual property licenses required for actual application. Telechips, Inc. does not provide any indemnification for any intellectual properties owned by third party.

Telechips, Inc. can not ensure that this application is the proper and sufficient one for any other purposes but the one explicitly expressed herein. Telechips, Inc. is not responsible for any special, indirect, incidental or consequential damage or loss whatsoever resulting from the use of this application for other purposes.

COPYRIGHT STATEMENT

Copyright in the material provided by Telechips, Inc. is owned by Telechips unless otherwise noted.

For reproduction or use of Telechips' copyright material, permission should be sought from Telechips. That permission, if given, will be subject to conditions that Telechips' name should be included and interest in the material should be acknowledged when the material is reproduced or quoted, either in whole or in part. You must not copy, adapt, publish, distribute or commercialize any contents contained in the material in any manner without the written permission of Telechips. Trade marks used in Telechips' copyright material are the property of Telechips.

Important Notice

This material may include technology owned by the 3rd party licensor and the technology may be subject to its associated licenses. It is solely customer's responsibility to identify and comply with such licenses. No other licenses are granted or implied by Telechips with making available this material.

For customers who use licensed Codec ICs and/or licensed codec firmware of mp3:

"Supply of this product does not convey a license nor imply any right to distribute content created with this product in revenue-generating broadcast systems (terrestrial. Satellite, cable and/or other distribution channels), streaming applications(via internet, intranets and/or other networks), other content distribution systems(pay-audio or audio-on-demand applications and the like) or on physical media(compact discs, digital versatile discs, semiconductor chips, hard drives, memory cards and the like). An independent license for such use is required. For details, please visit <http://mp3licensing.com>".

For customers who use other firmware of mp3:

"Supply of this product does not convey a license under the relevant intellectual property of Thomson and/or Fraunhofer Gesellschaft nor imply any right to use this product in any finished end user or ready-to-use final product. An independent license for such use is required. For details, please visit <http://mp3licensing.com>".

For customers who use Digital Wave DRA solution:

"Supply of this implementation of DRA technology does not convey a license nor imply any right to this implementation in any finished end-user or ready-to-use terminal product. An independent license for such use is required."

For customers who use DTS technology:

"This product made under license to certain U.S. patents and/or foreign counterparts."
"© 1996 – 2011 DTS, Inc. All rights reserved."

For customers who use Dolby technology:

"Supply of this Implementation of Dolby technology does not convey a license nor imply a right under any patent, or any other industrial or intellectual property right of Dolby Laboratories, to use this Implementation in any finished end-user or ready-to-use final product. It is hereby notified that a license for such use is required from Dolby Laboratories."

For customers who use Google technology:

"Copyright © 2013 Google Inc. All rights reserved."

For customers who use MS technology:

"This product is subject to certain intellectual property rights of Microsoft and cannot be used or distributed further without the appropriate license(s) from Microsoft."